

Reactive REST APIs with WebFlux (Functional Model)

So far we have been building reactive REST APIs using annotations, `@RestController`, `@GetMapping`, `@PostMapping` and so on. Spring WebFlux supports a second way to build the exact same APIs called the **Functional Programming Model**.

Two New Concepts

The functional model revolves around exactly two interfaces:

- `RouterFunction` - defines which requests go where. This is the replacement for `@GetMapping`, `@PostMapping` etc.
- `HandlerFunction` - defines what happens when a request arrives. This is the replacement for your controller method body.

By convention these live in two separate classes, a `Router` and a `Handler`. The router only knows about paths and HTTP methods. The handler only knows about request processing and response building.

The Router

The router is a Spring `@Configuration` class that exposes a `RouterFunction` bean.

```
@Configuration
@RequiredArgsConstructor
public class UserRouter {

    @Bean
    public RouterFunction<ServerResponse> userRoutes(UserHandler handler) {
        return RouterFunctions.route()
            .GET("/api/v1/users", handler::getAll)
            .POST("/api/v1/users", handler::create)
            .GET("/api/v1/users/{id}", handler::getById)
    }
}
```

```

        .PUT("/api/v1/users/{id}", handler::update)
        .DELETE("/api/v1/users/{id}", handler::delete)
    e)

    .build();
}
}

```

That is the entire routing config. No annotation scanning, no `@RequestMapping` on a class, no `DispatcherServlet` magic. Routes are evaluated top to bottom in the order you define them.

The Handler

The handler wraps `UserService` and builds `ServerResponse` objects instead of returning plain objects.

```

@Component
@RequiredArgsConstructor
public class UserHandler {

    private final UserService userService;

    public Mono<ServerResponse> getAll(ServerRequest request)
    {
        return ServerResponse.ok()
            .contentType(MediaType.APPLICATION_JSON)
            .body(userService.findAll(), UserResponse.class);
    }

    public Mono<ServerResponse> create(ServerRequest request)
    {
        return request.bodyToMono(UserRequest.class)
            .flatMap(this::validate)
            .flatMap(userService::create)
            .flatMap(created -> ServerResponse

```

```

        .status(HttpStatus.CREATED)
        .contentType(MediaType.APPLICATION_JS
ON)

        .bodyValue(created));
    }
}

```

Key differences from a controller method:

- Input is always `ServerRequest` instead of annotated parameters
- Output is always `Mono<ServerResponse>` instead of `Mono<T>` or `Flux<T>`
- You build the response manually with status, content type, and body
- Path variables come from `request.pathVariable("id")`
- Query params come from `request.queryParam("name")`

Reading Request Data

In annotation controllers Spring injects path variables and query params via `@PathVariable` and `@RequestParam`. In the functional model you read them from `ServerRequest` explicitly.

```

// path variable
Long id = Long.parseLong(request.pathVariable("id"));

// query param -- returns Optional
String category = request.queryParam("category").orElse("all");

// request body
Mono<UserRequest> body = request.bodyToMono(UserRequest.class);
Flux<UserRequest> body = request.bodyToFlux(UserRequest.class);

// headers

```

```
String authHeader = request.headers().firstHeader(HttpHeaders.AUTHORIZATION);
```

Request Validation

`@Valid` does not work anywhere in the functional model, there are no annotated method parameters for Spring to scan. You validate manually using a `Validator` bean.

```
private void validate(UserRequest request) {
    Set<ConstraintViolation<UserRequest>> violations = validator.validate(request);
    if (!violations.isEmpty()) {
        throw new ValidationException(toErrorMap(violations));
    }
}
```



Validation that is automatic in annotation controllers becomes a manual step you must remember to call in every handler method.

Error Handling

Option 1: `.onError()` scoped per route group

Handles exceptions inside the reactive pipeline before they ever escape the router. The recommended approach for the functional model.

```
@Bean
public RouterFunction<ServerResponse> userRoutes(UserHandler handler) {
    return RouterFunctions.route()
        .GET("/api/v1/users", handler::getAll)
        .GET("/api/v1/users/{id}", handler::getById)
        .POST("/api/v1/users", handler::create)
}
```

```

        // ... other routes
        .onError(ResourceNotFoundException.class,
            (ex, req) -> ServerResponse.status(HttpStatus
tatus.NOT_FOUND)
                .bodyValue(new ErrorResponse(ex.
getMessage()))))
        .onError(ValidationException.class,
            (ex, req) -> ServerResponse.status(HttpStatus
tatus.BAD_REQUEST)
                .bodyValue(new ErrorResponse(ex.
getMessage()))))
        .onError(IllegalArgumentException.class,
            (ex, req) -> ServerResponse.status(HttpStatus
tatus.BAD_REQUEST)
                .bodyValue(new ErrorResponse(ex.
getMessage()))))
        .build();
    }

```

Scoped only to the routes in this builder chain. If you have multiple `RouterFunction` beans, each needs its own `.onError()` chain.

Option 2: `@RestControllerAdvice` as a global handler

Works for both annotation controllers and functional routes since it sits at the `DispatcherHandler` level above both. Simpler if you want one centralized place for all error handling across your entire application.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public Mono<ResponseEntity<ErrorResponse>> handleNotFound
(ResourceNotFoundException ex) {
        return Mono.just(ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(ex.getMessage())));
    }
}

```

```

    }

    @ExceptionHandler(ValidationException.class)
    public Mono<ResponseEntity<ErrorResponse>> handleValidation(ValidationException ex) {
        return Mono.just(ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse(ex.getMessage())));
    }

    @ExceptionHandler(Exception.class)
    public Mono<ResponseEntity<ErrorResponse>> handleGeneric(Exception ex) {
        return Mono.just(ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("Something went wrong")));
    }
}

```

Option 3: Both together (recommended for production)

`.onError()` on each router for known business exceptions. `@RestControllerAdvice` as the catch-all safety net for anything unexpected that slips through.

```

// router -- handles known business errors
.onError(ResourceNotFoundException.class,
    (ex, req) -> ServerResponse.status(HttpStatus.NOT_FOUND)
        .bodyValue(new ErrorResponse(ex.getMessage())));

// GlobalExceptionHandler -- safety net for everything else
@ExceptionHandler(Exception.class)
public Mono<ResponseEntity<ErrorResponse>> handleGeneric(Exception ex) {

```

```

return Mono.just(ResponseEntity
    .status(HttpStatus.INTERNAL_SERVER_ERROR)
    .body(new ErrorResponse("Something went wrong"
g"")));
}

```

When to Choose Functional Over Annotation

Default to annotations. The functional model is a niche choice for specific situations.

The only real reasons to use functional:

- Dynamic route registration at startup, generating routes from a config or database
- API gateway style services with complex per-route filter logic that would be messy with annotations
- Per-route scoped error handling via `.onError()` where different route groups need genuinely different error strategies

Every other situation: use annotations.

- Same syntax as Spring MVC, zero learning curve
- `@Valid` works automatically, no manual validator injection
- OpenAPI and Swagger work out of the box
- `@RestControllerAdvice` handles all errors globally with no extra wiring
- Every Spring developer on your team already knows it
- Tooling, documentation, and community examples are all annotation-first



Both models produce identical runtime behavior, same Netty server, same non-blocking I/O, same performance. The functional model does not make your application faster or more reactive. If you cannot point to a specific problem it solves for your use case, use annotations.

